

HBase Cross Cluster Replication

Outline

- Theory of Operation
 - Overview
 - Log shipping
 - Sources and sinks
 - Arbitrary topologies
 - Load balancing
 - Delivery semantics
 - External requirements
 - Life of a WAL Edit
- Improvements and Future Efforts
 - Serial Replication
 - Casual Replication
 - Synchronous Replication
- Extensibility
 - The Endpoint API
- Applications
 - SEP
 - Org-directed replication



Theory Of Operation

RTFM

The best free resource for understanding cross cluster replication is the online manual:

http://hbase.apache.org/book.html#_cluster_replication

The design was proposed on [HBASE-1295](#). Of historical interest.

A note on “total” versus “partial” ordering

This presentation will use the terms *total order* and *partial order*.

There are formal mathematical definitions of these concepts, which I will not discuss in depth. Wikipedia has some introductory material:

- Total order: https://en.wikipedia.org/wiki/Total_order
- Partial order: https://en.wikipedia.org/wiki/Partially_ordered_set

Total ordering defines the exact order of every element in the series.

Partial ordering of elements in a series is an ordering that does not specify the exact order of every item but instead only defines the order between some items that depend on each other.

In the context of distributed systems, consider three events $\{ A, B, C \}$. They are totally ordered if they always have to happen in the order $A > B > C$. However, if A must happen before C, but you do not care when B happens, then they are only partially ordered. The sequences $A > B > C$, $A > C > B$, and $B > A > C$ all satisfy the partial ordering.



A note on “total” versus “partial” ordering (cont.)

In the context of cross cluster replication, consider the following set of keys:

$\{ A, B, C, D, E, F \}$

If all observers at the sink (destination) cluster see the ordering

$A > B > C > D > E > F$

then the replication stream is totally ordered.

If an observer at the sink might see orderings like:

$A > D > B > E > C > F$

$D > E > A > F > B > C$

(batches $\{A,B,C\}$ and $\{D,E,F\}$ are processed concurrently)

then the replication stream is partially ordered.

Overview

HBase provides a cross-cluster replication mechanism that keeps one cluster's state synchronized with that of another cluster, using the write-ahead log of the source cluster to propagate the changes.

Key properties

- Asynchronous log shipping optimized for performance
- Stochastic load balancing
- Support arbitrary replication topologies
- Eventually consistent (all edits eventually arrive)
- Deliver updates at least once
- Atomicity of individual updates is preserved
- Temporal ordering of updates is preserved by cell timestamp
- Data replication only, schema must be managed administratively

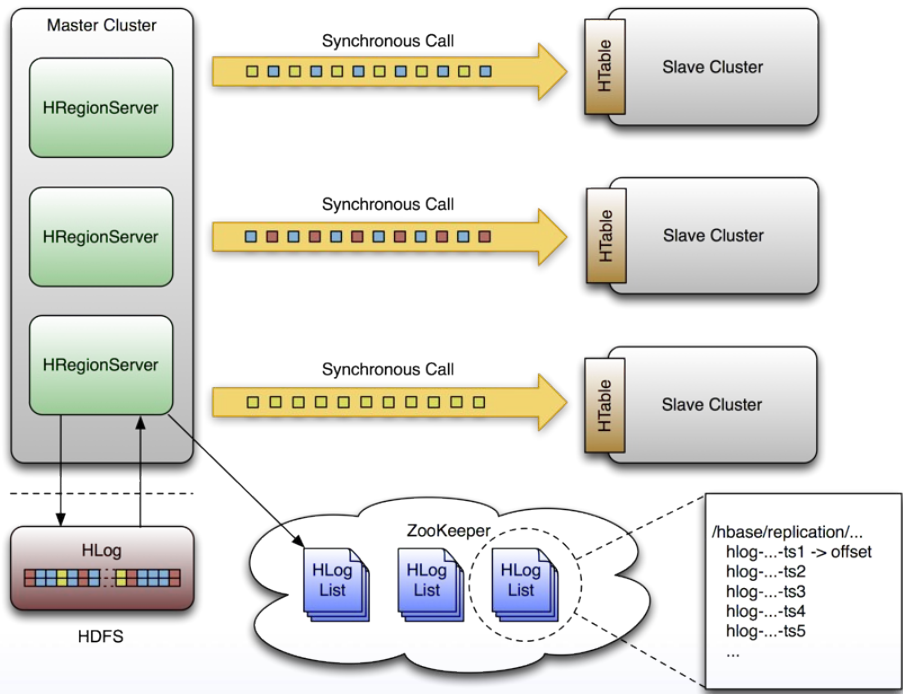
Log shipping

WALs are actively tailed. We don't wait for the file to be rolled. Edits are shipped as soon as they appear for read after HDFS write and immediate hflush or hsync.

WAL writers and readers can be different regionserver processes.

Edits are sorted by region and shipped concurrently in batches when batching is possible.

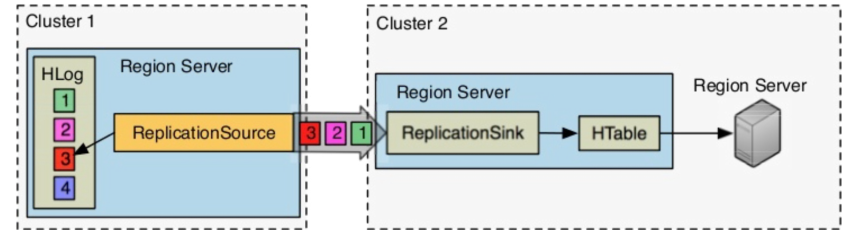
Queue status and position is durably tracked with strong consistency via ZooKeeper.



Sources and sinks

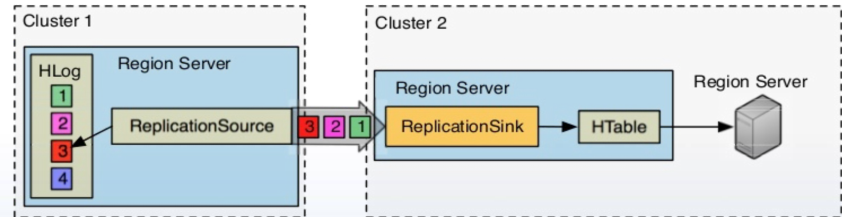
Replication source

- Endpoint for shipping WAL edits
- One queue for each regionserver instance (host+port+startcode)
- One source/thread for each queue
- Edits are filtered by replication scope, and batched by region
- Batches are synchronously shipped to sinks



Replication sink

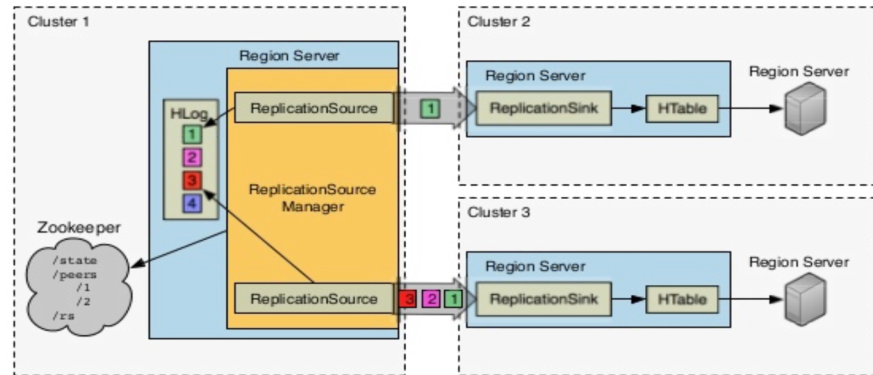
- Endpoint for receiving and applying WAL edits
- One sink instance per participating regionserver
- Applies edits atomically as a “normal” client
- Batch applies rows in the same table



Sources and sinks (cont.)

Replication source manager

- Watches the work queue in ZooKeeper for new sources to acquire
- Allocates a thread per acquired source
- Responds to changes in replication state:
 - Log rolling
 - Regionserver failure
 - Addition or removal of peer clusters



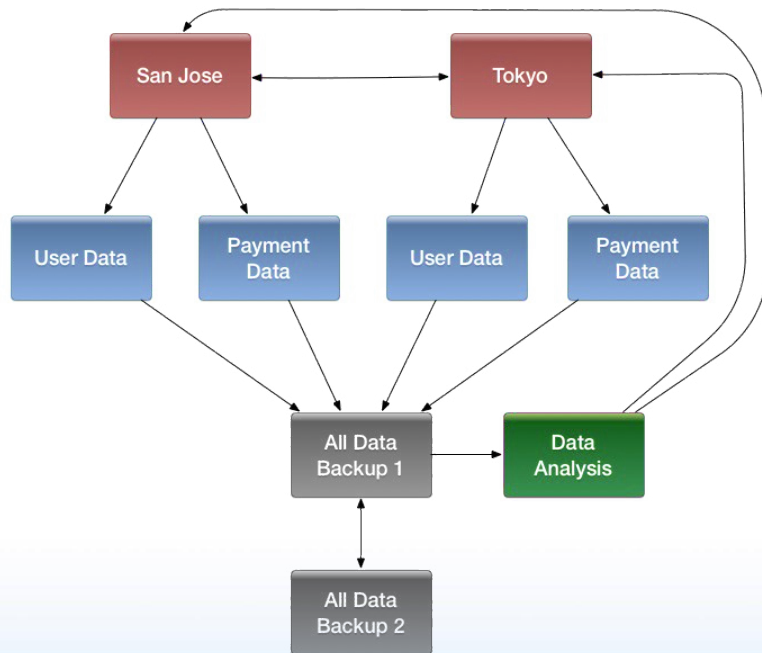
Arbitrary replication topologies

Replication topologies are defined by the `add_peer` or `remove_peer` commands, which add or remove one unidirectional edge to or from the replication graph. Peers are defined at site scope (per site). Arbitrary topologies can be defined, including loops.

- A central source cluster might propagate changes out to multiple destination clusters, for failover or due to geographic distribution.
- A source cluster might push changes to a destination cluster, which might also push its own changes back to the original cluster.
- Many different low-latency clusters might push changes to one centralized cluster for backup or resource-intensive data analytics jobs. The processed data might then be replicated back to the low-latency clusters.
- Multiple levels of replication may be chained together.

Arbitrary replication topologies (cont.)

An example complex topology from the [HBase online manual](#):

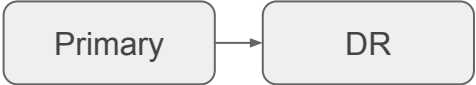


Each node on this graph is a separate cluster.

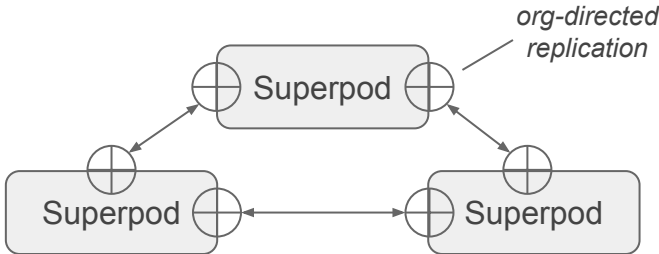
Each edge is a peer definition (or a pair of them).

Arbitrary replication topologies (cont.)

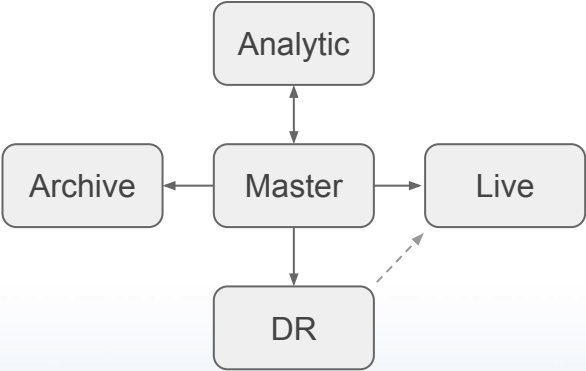
Other examples



Old SFDC Model



New SFDC Model



Wheel and spokes

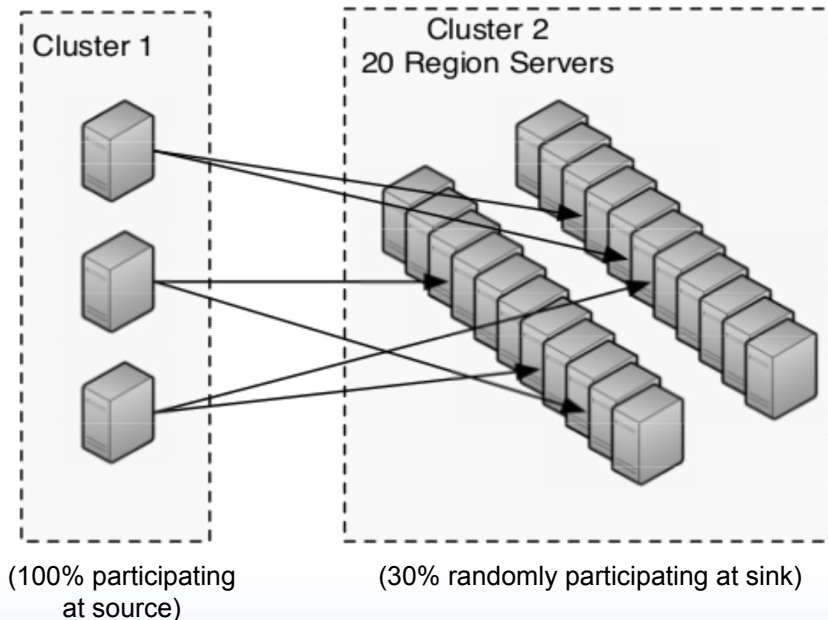
Stochastic load balancing and queue management

Only a subset of regionservers will register as sinks, decided randomly at startup. Size of the subset is a per cluster setting. Default is 10%.

At the source each regionserver replicates its own logs, unless there has been a failover, in which case the *recovery queues* are randomly picked up.

Recovery queues are assigned via a structure in ZooKeeper that ensures each queue is exclusively acquired. Failure handling posts affected replication queues for reacquisition.

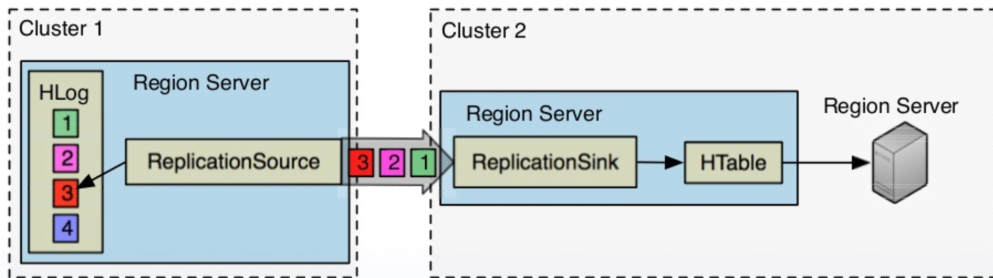
Sources choose sinks to connect to at random.



Batching, ordering, and delivery

Each replication source will ship one or more WALs acquired from a live regionserver. Edits in those WALs are grouped and ordered by region and shipped in batches.

In the simple case, all edits to regions hosted by a given regionserver are grouped by region and the batches are shipped concurrently for performance. (We don't want serial acking of each batch to pace the connection and limit use of the available network bandwidth.)

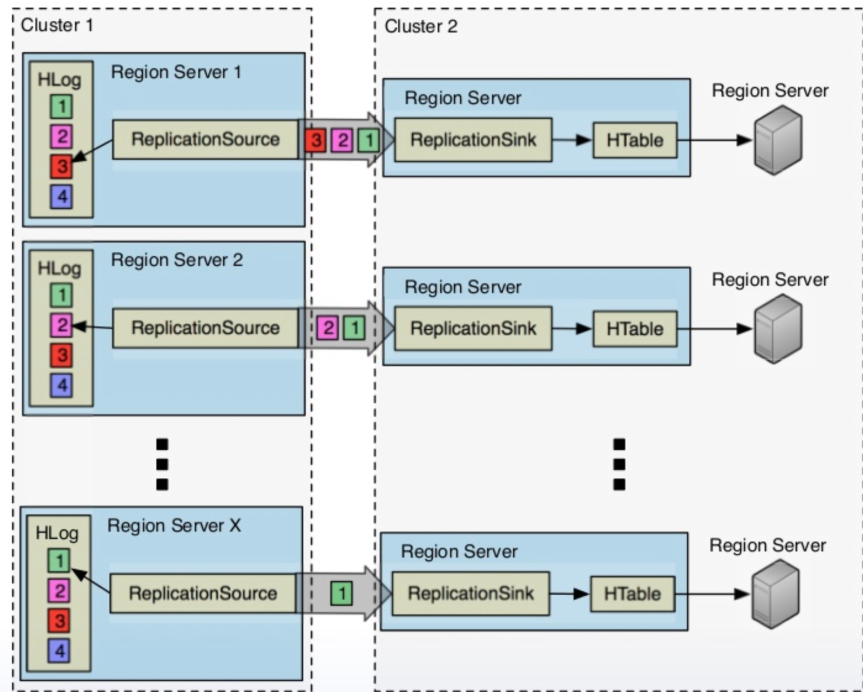


Batching, ordering, and delivery (cont.)

As regions move among regionservers, edits to those regions move among replication queues and may be processed in parallel by multiple replication sources.

Edits are delivered at least once. Arrival of edit batches may be interleaved and/or repeated or delayed by failure handling.

Temporal ordering is maintained at the sink using the timestamps of cells as committed at the source. Cell timestamps have millisecond resolution. Ordering at sub-millisecond scale is by key-type (deletes before writes).



Batching, ordering, and delivery (cont.)

Cross cluster replication by design **minimizes propagation latency**, **minimizes performance impact** to the source and sink clusters, and is **optimized for transport throughput** and dogged persistence despite the failure of any given source or sink regionserver, or transient issues in HDFS level block availability. As consequence:

- Edits are guaranteed to arrive at least once
- Edits are not guaranteed to arrive in any particular order
 - There is a partial ordering by timestamp and key type for each batch of edits for each region. This allows for micro-optimizations when applying the batches.
- Edits are not guaranteed to arrive in any particular timeframe (it is: as fast as possible)

Changes to support other ordering semantics for use by applications which desire to leverage or extend the replication subsystem, like change capture, are considered in later slides.



Batching, ordering, and delivery (cont.)

“Edits are guaranteed to arrive at least once”

Cases where we may deliver more than once:

- A WAL tailer encounters a HDFS level read error, like a read timeout or transient block availability issue (all replicas offline). The tailer will rewind to position 0 in the WAL file and begin again.
 - Consequence: The edits from the beginning of the WAL file forward may be replayed one or more times until finally we manage to reach EOF without errors
- A replication RPC, which encapsulates a set of changes to ship, fails after partial application at the sink. The failure can be at either source or sink. The source cluster will retry the entire set.
 - Consequence: The range of edits that were applied in the previous partially successful attempt will be applied again during the next attempt

Batching, ordering, and delivery (cont.)

“Edits are not guaranteed to arrive in any particular order”

Cases where we may interleave edits:

- A region moved from one regionserver to another
 - Consequence: Edits for the region appear in more than one replication queue, and replication queues are processed concurrently without fine grained coordination.
- A replication source failed
 - Consequence: The edits in the recovery queue will be transmitted concurrently without fine grained coordination while the edits in the new active queue are also shipped.
- Multiple replication RPCs are in flight
 - Consequence: Each batch contains a subset of edits to apply to a region. They are shipped concurrently by the source so may be processed concurrently at the sink.

Batching, ordering, and delivery (cont.)

“Edits are not guaranteed to arrive in any particular timeframe”

Cases where we may take a long time to ship an edit:

- The source cluster is degraded
 - Consequence: Queue backlog at the source. This is probably the best case because the sourcing application will also have some indication there is something wrong.
- The target (sink) cluster is degraded, a schema mismatch is present, or the network is degraded
 - Consequence: Queue backlog at the source
- Insufficient resources allocated at either source or sink
 - Consequence: Queue backlog at the source

ageOfLastShippedOp and other replication metrics can be used to determine if a backlog is present, but if that requires special handling, it is left to the operator and/or application to manage.

Batching, ordering, and delivery (cont.)

Application level consequences can be difficult to deal with:

- Replay, interleaving, and reordering of edit arrivals
- Put-before-delete: Data is visible at sink but is not visible at source
- Delete-before-put: Data is not visible at sink but is visible at source

Currently, when cross-cluster replication is active, application developers need to handle eventual consistency, even though HBase at the source cluster continues to provide strong consistency. *(That is, strongly consistent semantics remain available if data is written and read back only at the source cluster.)*

Common strategies assume timerange consistency. E.g. data is fully consistent from five minutes back to the beginning of time. Queries for latest data as expected may return inconsistent results.

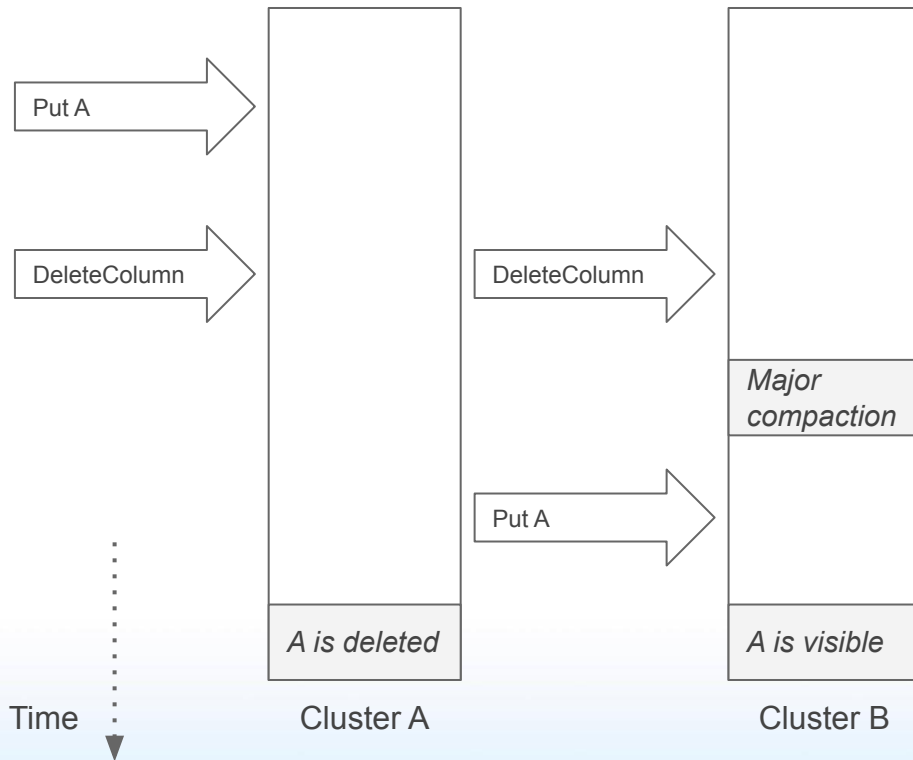
Replication metrics like `ageOfLastShippedOp` can be tracked to ensure reality matches expectation and switch to a degraded mode if not. We may want to provide an API enhancement that helps with this.

Batching, ordering, and delivery (cont.)

One pathological case of note. Say we have an out of order delivery involving Puts and Deletes:

- The Puts are delayed.
- Before the Puts arrive there happens to be a major compaction at the target. Any delete tombstones will be removed.
- Then the Puts arrive late. They would have been deleted if on time because they would have been covered by a tombstone.
- No tombstones now. The Puts will remain visible at the target.

To avoid this case we could leave tombstones in place for extra time at the target.



External requirements

At the sinks the received edits will be applied locally using the cell timestamp, which will have been set once at the originating source cluster.

In order for correct temporal ordering to be maintained across all clusters we have the following external requirements:

- All system clocks in all clusters should be synchronized with NTP.
- The NTP strata should implement [leap smearing](#), which avoids possible “time travel” when the clocks are corrected for a leap second.
- If the client is setting the cell timestamp the application developer must be aware that they are now manually managing temporal ordering and that the effective resolution for discriminating between moments is one millisecond. The client clocks must also be NTP synchronized.
 - We have a cache use case here at SFDC which did not do this and so encountered a corner case that failed cross cluster validation

Life of a WAL Edit

1. Source: An HBase client issues a write request to the local HBase cluster.
2. Source: The region server commits the request to the write-ahead log (WAL).
3. Source: The edit is read from the log by the replication source serving the replication queue. A WAL edit will contain one or more cells. Cells eligible for replication are processed. Others are dropped. Eligible cells will belong to a column family with a global replication scope, will not be in a system table such as *hbase:meta*, will not have originated from the sink cluster, and will have not yet been consumed (acknowledged) by the sink cluster.
4. Source: The edit is tagged with the master's UUID and added to a buffer. When the buffer is filled, or the reader reaches the end of the file, the buffer is sent to a random region server on the sink cluster. The regionserver sorts the edits sent by RPC into lists, one list per region. The lists are sorted by row key.

Life of a WAL Edit (cont.)

5. Sink: The regionserver reads the edits sequentially and separates them into buffers, one buffer per table. After all edits are read, each buffer is flushed using Table, the standard HBase client API. Every WAL edit carries a list of cluster UUIDs. The list will contain at least the source cluster's UUID. It will also contain the UUIDs of any other clusters in the topology that have consumed the data on the way to this replication edge. The UUID of the local sink is also added to this list. The list is committed with the cells of the entry into the local WAL. The sink cluster's WAL may itself be a replication source. In this way we detect and prevent loops.
6. Source: The offset for the last edit that has just been successfully processed is recorded in ZooKeeper.
7. Source: If the RPC fails, retry a configurable number of times, or then reselect a new subset of sink regionservers and resume retries. Resume with the current WAL file at the last successful offset.
8. Source: If a sink has failed, reselect a new subset of sink regionservers, and resume retries. Resume with the current WAL file at the last successful offset.

Life of a WAL Edit (cont.)

9. Source: If the outage is total, retry* until replication activity can proceed. Select a new subset of sink regionservers and resume retries. Resume with the current WAL file at the last successful offset.

10. Source: Meanwhile, WALs are rolled as normal. The WAL files will be tracked by the replication subsystem. WAL files may be archived but will not be deleted as long as any replication queue holds a reference to the file.

11. Source: When all edits in a given WAL file have been processed and acknowledged by the sink, the WAL file is removed from the replication queue. The WAL file is then eligible to be archived or deleted per standard housekeeping.

* - In the default configuration exponential backoff and retry activity will be exhausted after approximately 30 hours. If there is a total outage preventing replication activity exceeding this interval then the source will begin to drop queued WALs and an external process will be required to backfill data that was not shipped.



Improvements and Future Efforts



Serial replication (HBASE-20046)

Serial replication guarantees WAL files are processed in the same exact order as they are created in the source cluster. WAL edits shipped to the sink for a given region have the same exact ordering as at the source.

Ordering is preserved even when a region splits or is merged.

Ordering is guaranteed at region granularity. *(I think it is only a partial ordering because batches for a given region can be shipped in parallel. Need to check the code. If the per-region batch RPCs are serialized then there would be a total ordering per region.)*

For more details, see [HBASE-20046](#) or the [design doc](#) for the feature.

Available in HBase 2.1 or later. Backport to HBase 1 may be possible but would be involved.

You can try this out with our [technology preview binaries for 2.2](#) (2.2.1-sfdc-1).

Causal replication (HBASE-21856)

Casual replication is similar to serial replication but proposes that it is not necessary to process WAL files in strict order, which has expense and overhead to track. Instead, it will be sufficient to guarantee by other means that, for each region, the edits for the region arrive at the sink in order.

“It should be sufficient to do the following:

- Region servers only adopt the replication queues from other region servers for regions they (now) own. This requires log splitting for replication.
- Region servers ship all edits for queues adopted from other servers before any of their "own" edits are shipped.”

Likewise this can only provide a partial ordering at region granularity unless per-region batch RPCs are serialized. Then there would be a total ordering per region.

This proposed feature is not yet implemented.

Synchronous replication (HBASE-19064)

Synchronous replication provides a total ordering in cross cluster replication but trades off significantly against reduced flexibility in design of replication topologies.

From the design doc:

“The basic idea is easy to understand:

- Set up two clusters, active (A) and standby (S), and connect them with asynchronous replication.
- **All reads and writes are performed at A, S only receives replication data.**
- Besides the normal WAL logging, A will also write a copy of the WAL (remote WAL) to the HDFS cluster of S.
- When the asynchronous replication goes on, also delete remote WALs on S which have already been replicated.
- If A crashes and we want S to be the next active cluster, replay the remote WALs on S before offering service.”

For more information see [HBASE-19064](#) or the [design doc](#) for the feature.



Synchronous replication (HBASE-19064) (cont.)

Synchronous replication provides a total ordering at S of mutations committed at A. The WAL commit at A cannot happen until the remote WAL commit at S has also succeeded. Ordering is lazily assured, relying on WAL replay after switching the active role from A to S. (This is why active-active is not possible.) S always has the complete set of edits committed at A.

The synchronous replication feature has been committed to HBase master branch, which will eventually be the HBase 3 code line, but there is no release yet and master branch is not in a releasable state. An internal version is in production at XiaoMi. A variant implementation is in production at Alibaba.

There are no current plans to backport this feature to either the HBase 2 or HBase 1 code lines. If HBase 1 is the desired target, a backport to HBase 2 will be required first.

This feature is still undergoing active development, see [HBASE-20422](#).



Extensibility

The Endpoint API

[ReplicationEndpoint](#) is a plugin which implements the mechanism of cross cluster replication. When defining peers a specific or custom ReplicationEndpoint implementation can be specified in the configuration for each peer. In this way we can implement custom replication policy, transport mechanisms, and/or integration with third party systems and plug them in as additional replication edges in our overall replication topology.

ReplicationEndpoint is closely tied to [ReplicationSource](#) in a producer-consumer relation. ReplicationSource is an HBase-private class which tails the logs and manages the queue of logs plus management and persistence of all the state for replication. ReplicationEndpoint on the other hand is responsible for doing the actual shipping and persisting of the WAL entries in the other cluster.

The base abstract class [HBaseInterClusterReplicationEndpoint](#) should be used to implement the custom ReplicationEndpoint.

Implement [HBICRE#replicate](#) . You will be passed a [ReplicationContext](#) encapsulating the batch of WALEdits that need to be shipped. Return *true* if replication was successful, *false* otherwise, and if *false* the replication framework will take care of the retry.



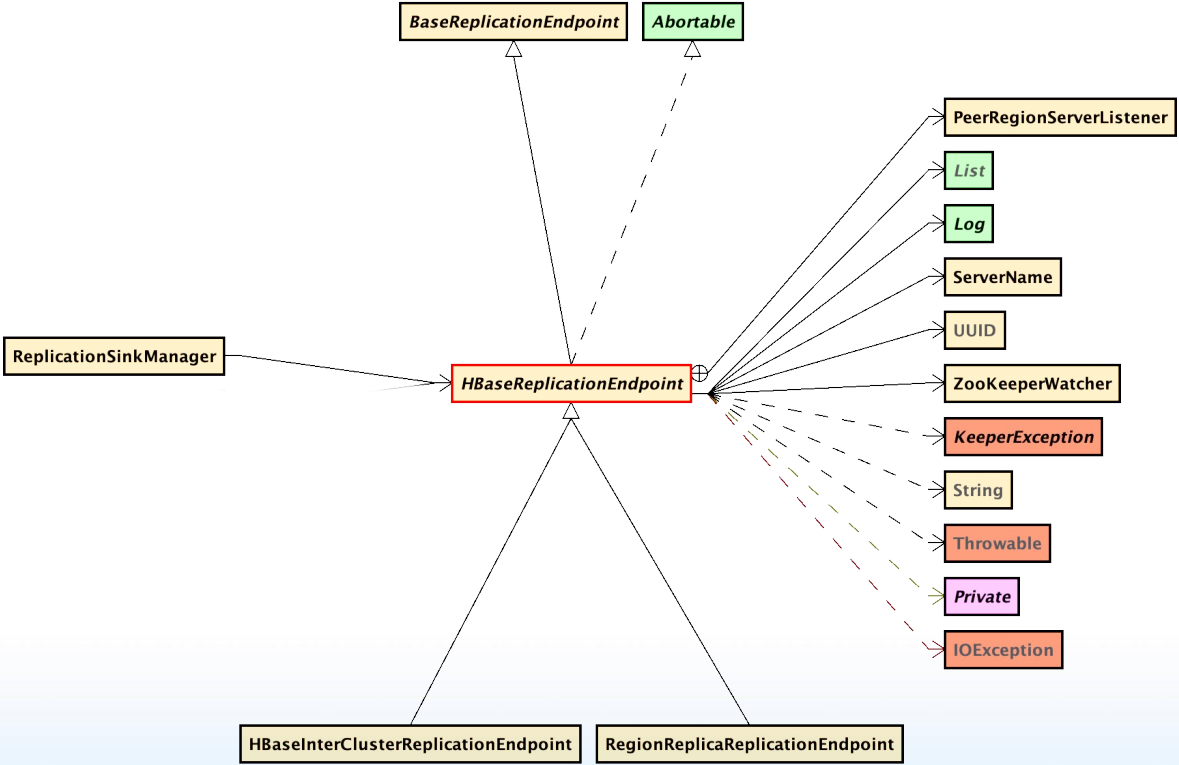
The Endpoint API (cont.)

Provide custom implementations of the various methods in [HBaseReplicationEndpoint](#) to manage the particulars of discovering peer clusters, discovering available endpoints in the peer cluster, and managing cross cluster connections.

At SFDC we are using the replication Endpoint API today to implement org-directed replication.

Plans for a Phoenix change capture device also center on the replication Endpoint API as integration point.

The Endpoint API (cont.)





Applications

hbase-sep

The HBase Side-Effect Processor ([hbase-sep](#)) is framework for external handling of mutation events asynchronously, originally designed for [NGDATA](#)'s Lily product. Now part of the Cloudera Search product. SEP implements to the HBase replication framework, and allows users to register external event listeners that will be passed a change stream from the HBase cluster in the form of cross-cluster replication RPCs.

Lily took this approach instead of using coprocessors when building a full text indexing solution so that the the distribution of events and execution of event handlers would have no direct impact on cluster write throughput (which is one of replication's goals).

A nice artifact of their implementation strategy is they can control the concurrency and ordering of sink side replication RPC handling (in theory) and so can derive a total ordering.

For more information see:

<https://www.slideshare.net/cloudera/ecosystem-session-4b>

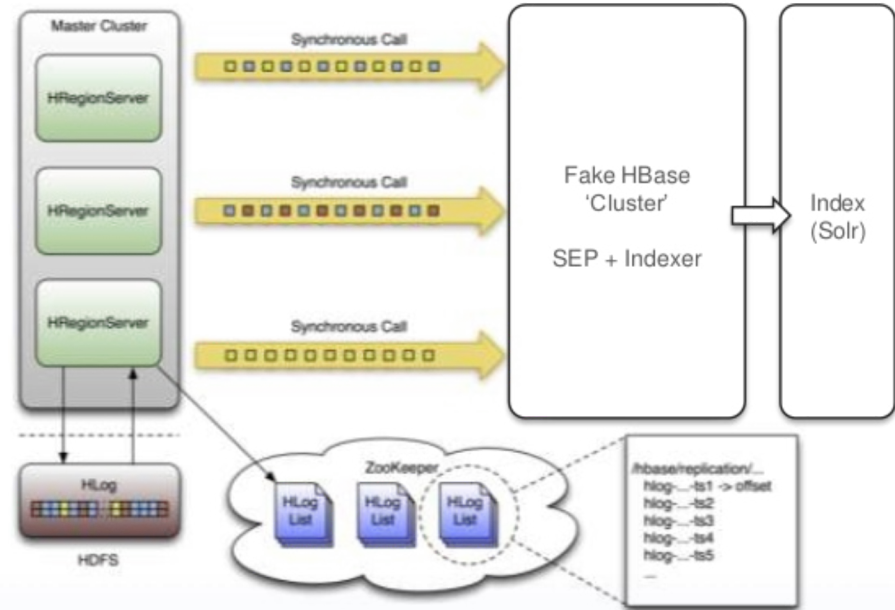
hbase-sep (cont.)

SEP implements enough of the regionserver RPC interface to accept replication RPC calls from a standard HBase source cluster.

As a library, it is embeddable.

The SEP “cluster” must run its own ZooKeeper (or share with the HBase source cluster). The SEP instance registers its endpoint there for the source cluster’s replication subsystem to automatically discover.

We can imagine an extension architecture where we add or remove replication edges to set up or tear down change data flows to independently scalable external applications.



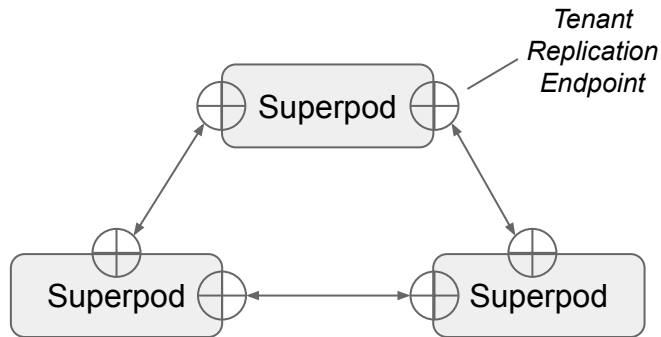
Org-directed replication

A logical pod is designated as primary and is hosted on one superpod. Another geographically separate superpod hosts the logical pod's DR buddy. Multiple pods are backed by superpods according to these constraints.

Replication edges are established between any superpod hosting a logical primary pod and the other superpod hosting the DR buddy.

A custom replication Endpoint implementation named `TenantReplicationEndpoint` filters the data that transits those edges to only what is required to service the hosted org(s). This is configured by a per cluster `TENANT_ROUTING` table.

For more details see [the Quip doc](#).





Questions?